

Finite stochastic kernels

Simon Frost

Table of contents

1	Overview	1
2	Setup	2
3	State spaces	2
4	Kernels and the axis convention	3
5	Sequential composition	5
6	Tensor product	5
7	Copy and discard	5
8	Copy once is not two samples	7
9	Summary	8
10	References	8

1 Overview

`FiniteKernels.jl` provides the executable semantics behind the `ecorecipes` compositional Bayesian-network packages: the category `FinStoch` of finite state spaces and stochastic kernels (conditional probability tables). A Bayesian network is a wiring of such kernels, and the operations that wire them together are exactly the operations of a *Markov category* ([Fritz 2020](#); [Cho and Jacobs 2019](#)):

operation	meaning
<code>compose_kernel(k, l)</code>	feed the output of <code>k</code> into <code>l</code>
<code>tensor_kernel(k, l)</code>	run <code>k</code> and <code>l</code> side by side
<code>copy_kernel(x)</code>	copy a value so several kernels can read it

operation	meaning
<code>discard_kernel(X)</code>	discard (marginalise out) a value
<code>swap_kernel(X, Y)</code>	swap the order of two wires
<code>identity_kernel(X)</code>	do nothing

This package names them without naming the category, and depends on nothing but `LinearAlgebra` and `Random`. `MarkovCategories.jl` states the same operations as a GATlab theory and registers them as a Catlab `@instance`, which is where they acquire the categorical names `compose` ($\cdot$), `otimes` ($\otimes$), `mcop` (Δ), `delete` ($\diamond$), `braid` (σ) and `id`; its vignette covers that layer.

This vignette introduces spaces and kernels, the axis convention, and the operations.

2 Setup

The package has no ecosystem dependencies; `Random` is used only for the reproducible random kernel at the end.

```
using FiniteKernels
using Random
```

3 State spaces

A `FiniteAxis` is one finite-valued variable: a name and an ordered list of unique state labels. A `FiniteSpace` is a product of axes; its joint states are enumerated in column-major order (first axis fastest).

```
rain = FiniteAxis(:Rain, [:yes, :no])
sprinkler = FiniteAxis(:Sprinkler, [:on, :off])
grass = FiniteAxis(:Grass, [:wet, :dry])

R = FiniteSpace(rain)
S = FiniteSpace(sprinkler)
G = FiniteSpace(grass)
RS = tensor_space(R, S)
```

```
FiniteSpace(Rain{yes,no}  $\otimes$  Sprinkler{on,off})
```

```
size(RS), length(RS), axis_names(RS)
```

```
((2, 2), 4, [:Rain, :Sprinkler])
```

```
joint_states(RS)
```

```
4-element Vector{Tuple{Symbol, Symbol}}:  
 (:yes, :on)  
 (:no, :on)  
 (:yes, :off)  
 (:no, :off)
```

The empty product is the monoidal unit I , a space with exactly one state. States (unconditional distributions) are kernels out of I .

```
FiniteSpace()
```

```
FiniteSpace(I)
```

4 Kernels and the axis convention

A `FiniteKernel` $X \rightarrow Y$ stores a table with the output axes first, then the input axes: `size(table) == (size(Y)..., size(X)...) , normalised over the output axes for every input index (ADR 0002). Interchange formats and the ecosystem's IR use the opposite, parents-first / child-last layout, so the user-facing constructor cpt takes that layout and performs the single documented permutedims.`

```
# P(Grass | Rain, Sprinkler) in the SPEC layout: (parents..., child), summing to 1  
over the last axis  
table = zeros{Float64, 2, 2, 2}  
table[1, 1, :] = [0.99, 0.01] # rain = yes, sprinkler = on  
table[1, 2, :] = [0.80, 0.20] # rain = yes, sprinkler = off  
table[2, 1, :] = [0.90, 0.10] # rain = no, sprinkler = on  
table[2, 2, :] = [0.00, 1.00] # rain = no, sprinkler = off  
k_grass = cpt([rain, sprinkler], grass, table)
```

```
FiniteKernel{Float64}(Rain{yes,no} ⊗ Sprinkler{on,off} → Grass{wet,dry})  
      yes,on  no,on  yes,off  no,off  
wet      0.99   0.9    0.8     0.0  
dry      0.01   0.1    0.2     1.0
```

Internally the child axis comes first:

```
size(k_grass.table), cpt(k_grass) == table
```

```
((2, 2, 2), true)
```

```
probability(k_grass, :wet, (:yes, :off))
```

```
0.8
```

Passing a table of the wrong shape, or one that is not normalised, is an error with a message naming the expected and actual sizes:

```
try
  cpt([rain, sprinkler], grass, rand(2, 2, 3))
catch err
  showerror(stdout, err)
end
```

KernelShapeError: cpt table must be laid out as (parents..., child) (expected size (2, 2, 2), got (2, 2, 3))

```
try
  cpt(rain, grass, [0.5 0.5; 0.7 0.7])
catch err
  showerror(stdout, err)
end
```

KernelNormalizationError: kernel FiniteSpace(Rain{yes,no}) → FiniteSpace(Grass{wet,dry}) is not normalised over its output axes (maximum deviation from 1 is 0.3999999999999999, atol = 1.0e-8)

kernel_matrix gives the column-stochastic matrix view (rows are output states, columns input states) that composition multiplies:

```
kernel_matrix(k_grass)
```

```
2×4 Matrix{Float64}:
 0.99  0.9  0.8  0.0
 0.01  0.1  0.2  1.0
```

Other constructors: state for a distribution, point_mass (or dirac) for a deterministic state, uniform, deterministic for a function on labels and random_kernel for tests.

```
p_rain = state(R, [0.2, 0.8])
k_sprinkler = cpt(rain, sprinkler, [0.01 0.99; 0.40 0.60]) # P(Sprinkler | Rain)
is_wet = deterministic(G, FiniteSpace(:Wet, [:t, :f]), g -> g == :wet ? :t : :f)
kernel_matrix(is_wet)
```

```
2×2 Matrix{Float64}:
 1.0  0.0
 0.0  1.0
```

5 Sequential composition

`compose_kernel(k, l)` is diagrammatic: first k , then l , with $(k \cdot l)(z \mid x) = \sum_y l(z \mid y) k(y \mid x)$. The domains must match.

```
p_sprinkler = compose_kernel(p_rain, k_sprinkler) # marginal distribution of
Sprinkler
p_sprinkler.table
```

```
2-element Vector{Float64}:
 0.322
 0.678
```

```
try
  compose_kernel(k_sprinkler, k_grass)
catch err
  showerror(stdout, err)
end
```

```
SpaceMismatchError in compose: expected FiniteSpace(Sprinkler{on,off}), got
FiniteSpace(Rain{yes,no} ⊗ Sprinkler{on,off})
```

6 Tensor product

`tensor_kernel(k, l)` runs two kernels independently on separate inputs; its table is the outer product with axes $(\text{codom}(k) \dots, \text{codom}(l) \dots, \text{dom}(k) \dots, \text{dom}(l) \dots)$.

```
two = tensor_kernel(k_sprinkler, is_wet)
two.dom, two.codom
```

```
(FiniteSpace(Rain{yes,no} ⊗ Grass{wet,dry}), FiniteSpace(Sprinkler{on,off} ⊗
Wet{t,f}))
```

```
probability(two, (:on, :t), (:yes, :wet)) == probability(k_sprinkler, :on, :yes) *
probability(is_wet, :t, :wet)
```

```
true
```

7 Copy and discard

`copy_kernel(X) : X → X ⊗ X` duplicates a value and `discard_kernel(X) : X → I` forgets it. They let one variable feed several kernels, which is how a Bayesian network's shared parents are wired. The sprinkler network $\text{Rain} \rightarrow \text{Sprinkler}$, $(\text{Rain}, \text{Sprinkler}) \rightarrow \text{Grass}$ is

```
# I --p_rain--> R --Δ--> R ⊗ R --(id ⊗ k_sprinkler)--> R ⊗ S --Δ(R ⊗ S)--> (R ⊗ S) ⊗
(R ⊗ S) --(id ⊗ k_grass)--> R ⊗ S ⊗ G
joint = compose_kernel(p_rain, copy_kernel(R),
                      tensor_kernel(identity_kernel(R), k_sprinkler),
                      copy_kernel(RS),
                      tensor_kernel(identity_kernel(RS), k_grass))

joint
```

```
FiniteKernel{Float64}(I → Rain{yes,no} ⊗ Sprinkler{on,off} ⊗ Grass{wet,dry})
()
yes,on,wet    0.002
no,on,wet     0.288
yes,off,wet   0.1584
no,off,wet    0.0
yes,on,dry    0.0
no,on,dry     0.032
yes,off,dry   0.0396
no,off,dry    0.48
```

The joint distribution sums to one, and discarding variables gives marginals:

```
sum(joint.table)
```

```
1.0
```

```
p_grass = compose_kernel(joint,
                        tensor_kernel(discard_kernel(R), discard_kernel(S),
                                      identity_kernel(G)))
p_grass.table
```

```
2-element Vector{Float64}:
 0.44838000000000001
 0.55162
```

marginal does the same by output-axis position or name:

```
marginal(joint, :Grass) ≈ p_grass
```

```
true
```

Discarding is *natural*: every normalised kernel followed by discard equals discard. This is precisely the normalisation condition, and it fails for an unnormalised table:

```
compose_kernel(k_grass, discard_kernel(G)) ≈ discard_kernel(RS)
```

```
true
```

```
u = FiniteKernel(R, G, [0.5 0.5; 1.0 0.5]; check=false) # first column sums to 1.5
compose_kernel(u, discard_kernel(G)).table, discard_kernel(R).table
```

```
([1.5, 1.0], [1.0, 1.0])
```

8 Copy once is not two samples

The defining feature of a Markov category, as opposed to a cartesian one, is that copying is not natural. For a state p , `compose_kernel(p, copy_kernel(X))` draws *once* and copies the outcome, whereas `tensor_kernel(p, p)` draws *twice* independently. Both have the same marginals but different correlation:

```
copied = compose_kernel(p_rain, copy_kernel(R))
```

```
FiniteKernel{Float64}(I → Rain{yes,no} ⊗ Rain{yes,no})
  ()
  yes,yes  0.2
  no,yes   0.0
  yes,no   0.0
  no,no    0.8
```

```
independent = tensor_kernel(p_rain, p_rain)
```

```
FiniteKernel{Float64}(I → Rain{yes,no} ⊗ Rain{yes,no})
  ()
  yes,yes  0.04
  no,yes   0.16
  yes,no   0.16
  no,no    0.64
```

```
copied ≈ independent
```

```
false
```

Equality holds exactly when p is a point mass (deterministic):

```
q = point_mass(R, :yes)
compose_kernel(q, copy_kernel(R)) ≈ tensor_kernel(q, q)
```

```
true
```

More generally `compose_kernel(k, copy_kernel(Y)) == compose_kernel(copy_kernel(X), tensor_kernel(k, k))` holds for deterministic kernels only:

```
compose_kernel(is_wet, copy_kernel(is_wet.codom)) ≈  
  compose_kernel(copy_kernel(G), tensor_kernel(is_wet, is_wet))
```

true

```
rng = Random.Xoshiro(1)  
k = random_kernel(rng, R, G)  
compose_kernel(k, copy_kernel(G)) ≈  
  compose_kernel(copy_kernel(R), tensor_kernel(k, k))
```

false

The tests in `test/test_laws.jl` check the full set of Markov-category laws (identity, associativity, interchange, comonoid and coherence laws, naturality of discard) numerically on random kernels.

9 Summary

A `FiniteSpace` is a labelled product of finite axes and a `FiniteKernel` is a conditional probability table stored outputs-first, with `cpt` as the single conversion to and from the parents-first layout that file formats use. Wiring kernels together needs only `compose_kernel`, `tensor_kernel`, `copy_kernel`, `discard_kernel`, `swap_kernel` and `identity_kernel`, and two of their properties already separate stochastic from deterministic maps: discarding is natural exactly when a kernel is normalised, and copying a single draw differs from taking two independent draws unless the state is a point mass. The companion package `MarkovCategories.jl` states these operations as a GATlab theory and evaluates symbolic string-diagram expressions in `FinStoch`; its “Markov categories as a GATlab theory” vignette continues from here.

10 References

- Cho, Kenta, and Bart Jacobs. 2019. “Disintegration and Bayesian Inversion via String Diagrams.” *Mathematical Structures in Computer Science* 29 (7): 938–71. <https://doi.org/10.1017/S0960129518000488>.
- Fritz, Tobias. 2020. “A Synthetic Approach to Markov Kernels, Conditional Independence and Theorems on Sufficient Statistics.” *Advances in Mathematics* 370: 107239. <https://doi.org/10.1016/j.aim.2020.107239>.